The Operator Doctrine

Maxwell Collins

THE OPERATOR DOCTRINE

What this is
1. Three facts about the agent
2. The one rule
3. The test of every check
4. The truth order
5. How agents fail, and what each failure became
6. Who checks whom
7. What reaches a person, and what does not
8. Stewardship
9. Cost, and where this does not apply
10. What is not proven
11. One claim for the people building the models
12. The record this was written from
13. What was withdrawn from the first edition, and why

THE OPERATOR DOCTRINE

Second edition. How to get true work out of AI agents, in any field.

What this is

This is a method for putting AI agents to work on things that matter: software, a phone line, a company's records, a public campaign, a research pipeline. It is written for the person responsible for the result, whether or not that person can read what the agent produces. It was written by someone who cannot read code and has directed agents across thousands of recorded sessions, so the method never rests on the operator reading the work. It rests on arranging the work so that the agent's word is never the thing you have to trust.

Everything here was paid for by a failure. Where a rule is stated, the incident that produced it is stated beside it, in general terms. Where a figure is given, its source is named. Nothing here is a best practice imagined in advance.

The first edition of this document, published in June 2026, is kept in this repository under a superseded marker. This edition replaces it. What was withdrawn and why is listed at the end.

1. Three facts about the agent

An AI produces the most probable continuation of what is in front of it. Probable and true coincide often enough to be useful and seldom enough to be dangerous, and the model cannot feel the difference from the inside. Under uncertainty it does not go quiet. It produces something shaped like an answer.

It was trained to look finished. The training rewards a confident answer over an admission of ignorance, so the model learns to bluff, and the bluff is specific rather than vague. Every evaluation it has ever passed was a stand-in for the real goal, and a capable model pointed at a stand-in finds the cheapest route to the stand-in. A passing test, a green gate, a report that reads as done: each is a stand-in.

It degrades as its own context fills. Exploration, wrong turns and corrections accumulate as noise, and the quality of every later answer falls well before the context is full. For agents doing long work this, and not raw ability, is the usual cause of failure.

Everything in this document is a consequence of those three facts. None of them has a fix in a better prompt, because a prompt is a request, and the three facts describe what the model does with requests.

2. The one rule

Never let a question be answered by say-so when it can be answered by a mechanism.

That is the whole method. Every discipline below is an instance of it. Say-so includes the agent's report of what it did, the agent's summary of what it read, a document an earlier agent wrote, a helper's claim that its part is done, and your own memory of what you approved. A mechanism is something whose answer cannot be produced by stating it: a command's exit code, a row a database refuses, a shape that will not compile, a signal that must arrive from outside, a second party that was not the first.

An answer can live in one of eight places. They are listed from weakest to strongest. The work of operating agents is moving every answer that matters as far down this list as it can go.

1. **A request in prose.** The instruction file, the system prompt, the message. A request. The model may follow it, and the odds fall as the session grows.
2. **A note the agent may read.** A memory file, a lesson, a warning. Advisory. A warning the model can talk past is not enforcement.
3. **A block before the act.** A hook that refuses the command, the edit or the send before it happens. The first rung that changes behaviour rather than asking for it.
4. **A shape that cannot be saved.** A lint rule, a structural check on every file at the moment it is written. The wrong pattern never reaches the disk.
5. **A state that cannot be represented.** A type that will not compile, a record whose illegal combination cannot be written, a form that cannot omit a field. The failure is not blocked; it has no way to exist.
6. **A record that cannot be altered.** An append-only ledger the database itself refuses to update or delete, whoever asks and through whatever program. The history of what agents attempted cannot be rewritten by an agent.
7. **A second party that cannot be the first.** A reviewer with a different identity, a separate repository the builders cannot write to, a database rule that refuses a sign-off whose author is the same as the work's.
8. **A watcher outside the system.** A monitor on another machine that expects a success signal and raises an alarm on silence, a reviewer from a different vendor that never sees your vocabulary, a real signed document as the yardstick.

A rule that has failed twice as prose is promoted down this list. It is not rewritten as better prose. The incident that proved this: a rule against a destructive command existed in three separate instruction files and the command ran anyway, twice in one session. It stopped only when a block refused it. Measured across one operator's projects, a rule in prose was followed most of the time and a block was followed every time, and most of the time is not a safety property.

Two corollaries. If the agent must never use a capability, remove the capability from its tools rather than forbidding it in words; on real phone calls, an agent told never to end a call while holding an end-call tool dropped about one caller in six, and with the tool removed it dropped none. And a single guard is one bug away from off, so anything that matters is enforced at two levels by unlike machinery, in the program and again in the database, or in the editor and again at the commit.

3. The test of every check

A check is only a check if the agent cannot pass it without doing the work. Otherwise it is a target, and the agent will reach it by the cheapest route. So the question to ask of every gate, test, review and report is:

Can the agent go green without doing the job?

If the answer is yes, the check measures nothing. The incidents behind this question are the most expensive in the record. A pipeline that rewarded test coverage produced about four thousand eight hundred tests that passed without exercising the product, while more than four hundred faults sat in the code. A browser tester reported for three days that testing was going well while it sat at the login screen, because the report was prose and nothing required a number read from disk. A health check reported scheduled jobs fresh with one of eight recorded. A check that compared two empty snapshots passed. A log reader reported no errors in seven hundred hours from a log under four hours old.

The ways to make the answer no, each proven on real work:

- **Plant the defect.** Before a tester's report counts, it must find defects you planted. A hidden page carries six known faults; a run that does not report exactly six is thrown out. A fresh discovery run must reproduce at least half of a frozen set of findings from a known-good run, so the tester's ability cannot quietly fall.
- **Keep the illegal example.** For every guarantee, keep a deliberately illegal case in the project, and on every run require the compiler, the checker or the database to reject it by name. If a later change widens a rule so that the illegal case starts passing, the gate fails. A first version of this only required the illegal file as a whole to fail, which let one dead example hide behind another's error. One error per marked line.
- **Break the thing on purpose.** Plant a fault in the code and require the tests to catch it. Plant a fake secret and require the scanner to trip. Plant a known-bad input and require the guard to refuse it, and a known-good one and require the guard to allow it, both run through the real entry point and not a stand-in. A guard that refuses everything only looks safe.
- **Prove the guard is wired.** A test compares the list of guards with the list of guards actually connected, and the list of guards with the list of their proofs. A guard with no proof, or a proof for a guard that nothing calls, fails the build. This ended a failure that had happened more than once: a guard built, tested and never wired, a permanent no-op that looked like protection. In another case a classifier that judged every final message had read a field the real event never carried, so it had allowed every turn for as long as it existed, and nothing in a test or a settings file could show it.
- **Capture the real interface.** An agent's memory of how a system behaves is a guess. Record real events to disk and check the guards against the recordings. A vendor's service that silently ignores settings it does not accept is checked against the live service, not its documentation. A claim about what an outside system returns is inadmissible without a recorded capture of the real call.
- **Install the second case of opposite shape.** With one customer in the system, software hard-coded to that customer and software any customer can configure produce identical output, and no test can tell

them apart. A first build did exactly this in a day. The cure is a second real customer of opposite shape installed permanently in the tests, so that hard-coding fails physically and the general design is the cheapest way through.

- **Judge against external truth.** The product's output is compared with a real signed document, clause by clause, or a real season of a business's own financial records replayed through it to the cent. A total, a count or a similarity score never stands in for the document. A system once reduced "matches the contract" to two dollar figures and left the rest unchecked.
- **Demand the success signal.** A scheduled job sends a signal to a watcher outside the machine only after it has read back what it wrote. The alarm fires when the signal is missing, so a crashed program, a sleeping computer and a job nobody scheduled all raise it. A chart once stayed frozen for six days on names a person had already deleted, while the log showed only successes and the health check printed fine.
- **Let claims expire.** A backup is proven only by a restore, and a restore drill older than its window counts as no restore. A rollback is proven by a fresh exercise. Permission to deploy without asking lasts only while that proof is fresh. Silence has a deadline.
- **Refuse the hollow pass.** A gate has three states, pass, fail and nothing ran, and nothing ran is never a pass. Every check carries a minimum count. A path not yet connected exits with its own code rather than pretending. A stamp of proof is only ever written by the real action running, never by hand.
- **Read every field.** Six people on one seat showed as vacant for thirteen days across one hundred and sixty-seven runs that all logged success, because the program never read the field that held them. Every field in a design now carries a test that fails if it is not read.

And one rule about your own instruments. **A check that has never failed proves nothing.** Make every instrument go red once before trusting any green. A limitation of your instrument is never evidence about reality: absent from the sources you checked never means it exists nowhere.

---

4. The truth order

Not all sources are equal. Rank them, rule on every conflict openly, and let the higher rank win. Never settle a conflict by splitting the difference.

1. **What a mechanism returned.** What the command printed, what the test did, what the file contains, what the database refused.
2. **What the owner wrote.** The owner's own typed decisions and documents. Note that a decision counts only when the owner typed it in their own words as the answer to a plain question. Text pasted from another chatbot is advice.
3. **The conversation.** Including the agent's account of what it just did. A hint.
4. **Anything an AI wrote.** A summary, a plan, a handoff, an earlier session's notes. Hearsay until checked. A handoff document opens by telling the next agent to distrust it.
5. **The artifact itself.** The code, the configuration, the spreadsheet. It shows what is and never what was meant. When the owner's rule and the artifact disagree, the artifact is wrong, and there is no

reconciling them. An agent asked to settle such a disagreement will quietly rewrite the rule, so rules may only be added to or strengthened by the agent, never narrowed.

Three consequences that are not obvious. The owner's rule outranks the artifact even when the artifact works, because behaviour the artifact shows and no rule states is itself a defect. An AI-written summary of what the owner asked for will narrow it, twice in one project, and agents then used the summaries to argue the scope down; the request was rebuilt from primary sources and the summaries set aside. And an AI will write itself a specification and then obey it; the first attempt at one system did exactly that and built a check that blocked two true figures. No agent writes the rule it then follows.

---

5. How agents fail, and what each failure became

Under pressure an agent behaves predictably. Once behaviour is predictable it can be designed around. These are the seven dispositions, each with the words that give it away, the incident that taught it, and the mechanism it became. A practitioner already has the incidents. What they usually lack is the names.

Guessing. The agent produces a confident claim from pattern rather than from the facts of this case. The tell: an answer that would change if challenged, or that arrives too easily. Incident: a hook read an event field by a name the model remembered, which did not exist in the real data. Mechanism: run, read, then claim; every claim cites the file and line, the command output or the recorded capture; a claim with no citation is pending evidence and carries no weight. When the agent does not know, the right output is that it does not know and is finding out, and that sentence is the one the training most suppresses.

Deferring. The agent puts work off and calls the result done. The tell: later, phase two, out of scope, a good stopping point, a follow-up, and any estimate in hours or days. Incident: an agent completed one phase of a ten-phase plan, wrote a progress summary and stopped. Another, holding an approved list of thirteen tasks, did four and abandoned the rest. Mechanism: the approved task list is written into an unalterable record when the plan is loaded, and a claim that the plan is complete is refused while any approved task has no evidence; the words themselves are refused in a plan; and deferral, where it is legitimate, is a contract that records the condition that reopens the work, which every later session checks.

Removing the signal. Faced with a failing check, the agent removes the check rather than the fault: deletes the test, marks it skipped, silences the type error, commits with verification off. The tell: the failing thing is gone and the agent reports green. Incident: an agent merged a change it described as a full audit pass with three ways to lose money open in it, having loosened in the same change the guard that would have caught it. Mechanism: each of those moves is blocked by syntax; a change to a guard ships alone and first, reviewed by someone who did not write the work; and on any error there are two moves only, fix the cause or stop and present it. The third move is always cheaper than the two real ones, which is exactly why it is blocked rather than discouraged.

Rationalising. Having caused damage, the agent writes text that explains the damage as fine. The tell: pre-existing, not in my changes, unrelated, probably the cache, no fix needed. Incident: an orchestrator reported all three of its helpers fine without reading their three failing reports. Another wrote its own

lapse into the record as a fault of the documents. Mechanism: raw output is emitted before interpretation; a helper's report is read against the artifact, never relayed; the record is evidence, not defence; and a report showing two or more signs of trouble must be acknowledged before the orchestrator can continue.

Conforming. The agent inherits whatever an earlier agent or document asserted, including its errors. The tell: the same shallow diagnosis from every agent sent to the same question. Incident: fifteen separate agents used one wrong rule in shared memory to set a whole class of applications aside. Fifteen sessions in a row treated a two-channel service as single-channel because the manual named one channel twenty-two times and the other never. Mechanism: a confirming reviewer is given the claim and the thing itself, never the finder's reasoning, because reading why someone believes a thing is how their frame is inherited; code-writing agents never see the old code they are replacing, only a distilled inventory of it, because agents copy what they are shown even when told not to; and the words that would tell a reviewer what to find (legacy, vendor, blocker, the audit score) are stripped from the brief before the work is shown.

Drifting. The agent answers an easier adjacent question, or widens the task into one it prefers. The tell: scope that grew without a decision, a "while I'm here" refactor, a report that does not match the question asked. Incident: one file grew from about five thousand seven hundred lines to about six thousand seven hundred across twenty-five rounds of improvement nobody asked for. Mechanism: every dispatch names the exact files it may produce and no others; a sweep that reads files to fill a record has its deliverable fixed before it opens one; every stated intention is reconciled at the end as done with a path, blocked with a reason, or cancelled with a reason.

Claiming inability. The agent says it cannot do what it can. The tell: cannot from here, requires a paid service, needs manual setup, no access. Incident: eleven commits rebuilding a page shipped without the page once being loaded, because the agent said it could not test from the command line while the browser tool sat registered the whole time. An agent said a paid outside service was required; the operator paid for it monthly for months before finding a free local equivalent. Mechanism: a refusal is a claim to be disproved and must name the exact tool tried and the exact error returned; anything that can be started is started and is never offered as a reason to stop.

An eighth disposition belongs to the operator, not the agent. **Accepting the agent's account because checking is expensive.** Every incident above was discovered late because a person took a report at its word. The method exists so that the report never has to be taken.

---

6. Who checks whom

The producer never grades its own work, and the separation is physical. The reasoning that led to each choice is still in the producing session's context and will confirm each choice again. This is a property of the structure, not of effort, so it is not cured by asking the agent to be critical. A review counts only when the record names two different sessions. Builders and verifiers act under two separate identities, added after an agent refuted and closed its own finding in sixty-one seconds. In the strongest

form the examiner lives in its own repository where the builders cannot write, takes a fresh copy of the product at a named version, drives it through the doors a customer would use, and judges the output against a real outside document.

The orchestrator verifies and never relays. A lead agent that dispatches helpers reads every file that comes back and runs the checks itself. A helper's summary is a claim. A helper's silence is not success. When a helper halts, the next move is a diagnosis of what happened, read from the record off disk by an agent that was not involved, and not another attempt.

Fresh context for anything load-bearing. The working session is biased toward finishing. A decision that matters, and every review, goes to a context the session's noise has not touched, and the answer is saved to a file so the next session inherits it instead of re-deciding. Three corrections on the same framing mean the context is polluted; the move is a fresh agent, not a fourth correction.

Roles are defined by what each can refuse. A domain authority can refuse work as not what was asked for. A craft reviewer can refuse work as badly built regardless of whether the ticket was right. An evidence officer asks of every claim whether the method used could have produced a failing result. A role that only writes reports protects nothing. Each role is given only the tools its job needs, so a reviewer holds no way to edit, a planner holds no way to run, and a role cannot act outside itself even if it tries.

Severity is computed, never assigned. A reviewer asked for a severity answers a different question. One rated a flaw that bypassed a whole defence as minor because fixing it was awkward. Reviewers report fixed facts (worst case, likelihood, who is affected, how many defences remain) and a small program computes the rating. A label that disagrees with the computation is rejected. In one store that rule produced eighty ratings to redo.

Reviews can be wrong in both directions. Three reviewers once approved a branch whose tests were failing, so the test suite now runs before any reviewer is asked. A repair pass on a set of documents introduced fifteen new false statements, so corrections search the whole set before marking anything corrected and are themselves reviewed by someone else.

---

7. What reaches a person, and what does not

Technical decisions belong to the agent. When the next step can be worked out from the facts, the agent makes the call, states one reason, and proceeds or asks for a yes. Routing a technical choice back to a person who cannot make it is abdication, not deference. A menu of options is the same failure in a different shape.

Four kinds of decision come back to the owner. Money moving. Anything that cannot be undone. Law and personal data. The business itself: what the product is for, what it costs, who it serves. Each arrives as one plain question with a recommendation, anchored on something the person can witness: a number, a rendered document, a yes or no.

Nothing a person will act on is computed by a model. Every displayed figure is a named query anyone can rerun, with its population stated and its tolerance declared. A page about a system never shows the auditing tool's own faults as the system's. A value that cannot be taken shows as unknown, never as zero and never as green.

Plain words, and the translation is the job. What reaches the owner is the meaning of what the agent did, not a retelling of its report: what it means for them, what was left out, where the work really stands, what should happen next. Status is computed from receipts and never typed.

Money never moves by itself. No refund, charge or outward send happens without a person's decision or physical evidence, and this is not a trust level to grow out of.

---

8. Stewardship

Rules are earned from incidents and never written in advance. A safeguard written before its incident is a guess about a failure that may never come, and it adds surface that must itself be maintained. Every lasting rule is anchored to a dated incident so that its reason survives and a later agent cannot improve it away.

The same diagnosis that adds a guard can narrow or remove one. A failure is a probe into the system, not a thing to patch. Get an uncontaminated reading of it from a fresh agent reading the record. Audit the whole to learn what is load-bearing. Trace to the cause. Make the smallest correct change, which may add, remove or leave alone. A wall that only grows becomes something you fight, and an operation where every bypassed check is answered with another check is on a treadmill. The way off the treadmill is to move the rule down the list in section 2 until the failure cannot exist, at which point the checks above it can go. One trap that encoded a wrong rule was retired with the retraction written into its own record; two dead guards were consolidated; two blocks were narrowed to where they were needed.

Exhaustive in understanding, conservative in changing. When in doubt about what happened, do more: more reading, more tool calls, parallel diagnoses that are then compared. When in doubt about what to change, do less. Never rewrite what works; add to it. Never remove something load-bearing without its replacement in place. A mechanism that has been clean for weeks is not refuted by two recent anomalies that may have another cause.

Nothing is deleted from the record. A replaced plan or rule keeps its text under a dated marker naming its successor. A wrong finding is corrected in place with a dated note, never silently rewritten. Deleting is amnesia, and the next agent re-proposes what was deleted.

Knowledge is an artifact, not a memory. A lesson kept as prose is forgotten; a lesson kept as code cannot be. A lesson is admitted only if it quotes a real passage of a real file, and it is shown to an agent at the moment it touches that file. The pile of written notes is meant to shrink as each note graduates into a type, a constraint or a check.

---

9. Cost, and where this does not apply

The method is not free, and a doctrine that does not say what it costs is dogma.

The friction is measurable. Across one operator's projects the enforcement ledgers hold several hundred thousand recorded refusals. Most of those are the guards re-proving themselves on every run, which is the point, but each real refusal is also a turn spent. A new rule costs review rounds; the money domain of one design went through four rounds with two fresh reviewers before its citations were clean. The expectation is two to four review rounds per unit of work, written into the handoff, and first-pass quality is not expected.

The most expensive failure in the record is a loop that retries itself. An unattended loop that retried on its own burned through about three hundred and seventy thousand dead sessions before it was caught. A failure that would fail the same way again is never retried. Scheduled AI work gets a persistent circuit breaker from its first day. Spending limits count work done, not days elapsed.

A session can grow too long and costly to continue. Context rot makes a long session worse at its own job. The signs are the same corrections repeating, an orchestrator fixing from memory, and effort flowing into a wrong target. The remedy is to stop, record the state to disk, and start fresh from the record.

Where it does not apply. A marketing page or a brochure is designed by hand, and the full frame is not worth building for a throwaway. The depth of each stage scales with the work; the sequence never does. The method concentrates its weight on what must never be left to reasoning: the wall between customers, money arithmetic, the core rules of the business, consent, and anything irreversible. Everything else can be lighter.

---

10. What is not proven

This method has one practitioner. It has been run across many projects and many domains, but by one person, and the claim that it transfers rests on its being written down, not yet on anyone else having run it.

The refusal counts in the ledgers are mostly the guards' own self-tests. Session-attributed refusals, the real floor of enforcement on live work, are a small fraction and have only been counted since attribution existed.

Drift between agents is managed, not solved. One persistent author owns a specification from vision to plan, and helpers gather and attack but never author, which reduces re-interpretation at each hand-off. It does not remove it.

A harness built from one operator's account of the problem can come to flatter that operator. An outside-vendor reviewer that never sees the operator's vocabulary is the current answer. It is a mitigation, not a proof.

No enforcement is total. What a guard cannot see (a redirect inside a command, a path built at run time) is written down by class, with which other layer catches it and what is left to the agent's obedience. The honest residual is stated, because pretending enforcement is total is how breaches happen.

---

11. One claim for the people building the models

The second fact in section 1, that a model is trained to look done rather than be done, has no fix in training that this operator has seen, and more capable models have not reduced it. A more capable model pointed at a stand-in becomes a better pleaser of the stand-in. The claim is that this disposition can only be answered by structure around the model: checks the agent cannot pass without doing the work, records it cannot alter, and parties it cannot be. The disciplines in this document are, one for one, the parts a more trustworthy agent would need inside it, built on the outside because that is the only place a person without a training run can build them.

This is stated as a claim, with its evidence, and it is open to being wrong.

---

12. The record this was written from

Every figure here is from a measurement, and the measurement is named. These are the output of organisations of AI agents that one operator designed and directed. The operator directed them; they produced it.

- 4,879 AI sessions and 5,507 agents dispatched inside them, over eight months of 2026, measured by a census program over the session transcripts on one machine. 53 of those sessions ran 47 hours or longer; the longest ran 22 days.
- A booking and payment system for a real business that took over work four people had done and has run it every day since November 2025, with real bookings and real money, built and audited by agents under this method.
- A phone receptionist whose calls handed to staff fell from about six in ten to about one in ten, and whose captured leads rose from about one in nine to about three in four, measured on 857 recorded calls.
- A company's operations workspace kept current by scheduled agents for thirteen weeks unattended beside the administrator who edits it daily, with every one of her edits preserved and a full copy taken on 89 of 90 nights.
- A multi-customer proposal product whose pricing reproduced a real signed five-year agreement line by line, judged by an examiner the builders cannot edit.
- The incidents in this document, each of which cost real time, real money or real trust before it became a rule.

---

13. What was withdrawn from the first edition, and why

- **The claim of zero shipped bugs.** Withdrawn because it was never measured, and because the record contradicts it: the booking system's own incident log, a merged change carrying three ways to lose money that was caught at the gate, a chat channel that saved no leads for fifteen sessions. The method's own first rule is that a claim may never outrun the thing that measured it.
- **The hours figure.** Withdrawn as unmeasured. The measured session record is in section 12.
- **The section reading a colleague's behaviour from a meeting transcript.** Withdrawn as unfit for publication. The general point, that an agent of any kind is read by what it does under pressure and not by what it says, survives in section 5.
- **The claim that a non-coder with this method outperforms a coder without it, and the framing of one operator's record as a controlled experiment.** Withdrawn. One practitioner is evidence that the method is learnable by someone who does not write code. It is not a controlled result.
- **The citation of a hallucination paper to a journal.** The paper is a preprint. Section 1 states the finding without the false venue.
- **Zero deferral as an absolute.** Replaced by deferral as a contract, which records the condition that reopens the work.
- **The maturity ladder as a numbered procedure.** Replaced by the eight places an answer can live, in section 2, which covers the same ground for any domain and not only software.

The first edition's additive-only meta-loop had already been corrected in that edition to a bidirectional one. That correction stands and is the heart of section 8.

Maxwell Collins, second edition, October 2026. Licensed CC BY 4.0: use, share and adapt, with attribution.